

AWS Python Deployment

Session 2 — Serverless Deployment

AmaliTech · Python Chapter

2025

Contents

Before You Start

Demo 1 — Your First Lambda + API Gateway

Demo 2 — Event-Driven Image Thumbnailer

Demo 3 — Full CRUD REST API with DynamoDB

Cleanup — Delete everything we created

Section A — CI/CD with GitHub Actions (optional, ~15 minutes)

Before You Start

What you need on your laptop

- An AWS account where you can create resources
- A web browser to use the AWS Management Console
- A terminal on **Ubuntu Linux** (every shell command works as-is)
- Python 3.12 installed for local testing
- A free GitHub account (for Section A only)

What we will deploy

Three serverless mini-projects, each shorter and simpler than Session 1's:

- **Demo 1:** a single Lambda function exposed through API Gateway — a "hello" API
- **Demo 2:** an event-driven image thumbnailer (S3 -> Lambda -> S3)
- **Demo 3:** a complete CRUD REST API backed by DynamoDB

We use the **AWS Management Console** for every step. All resources go in `eu-west-1` (Ireland).

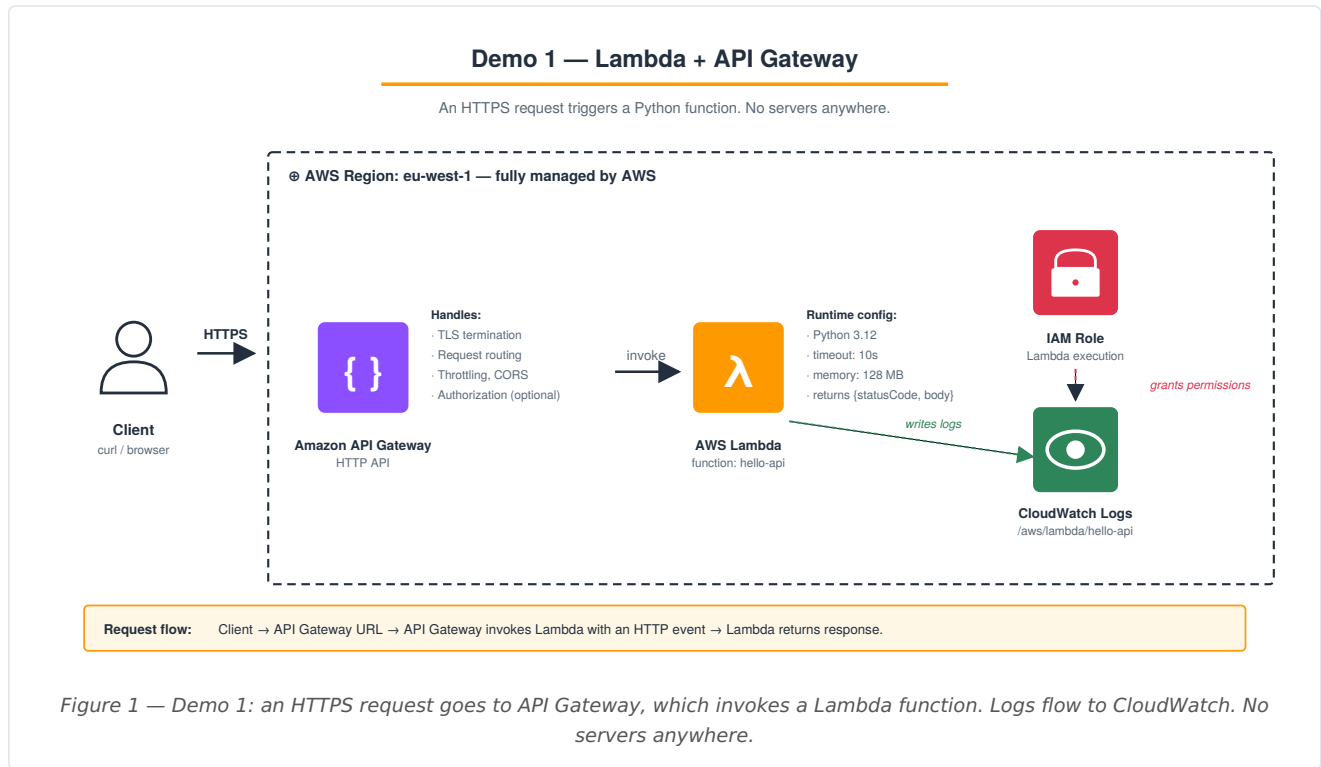
Tip: tag every resource with `Project = python-chapter-demo` so cleanup is easy. Most "Create..." forms have a Tags section near the bottom.

What "serverless" means

You upload code. AWS runs it on demand. You never log into a server. You pay only for actual execution time — when no one uses the system, you pay zero.

Demo 1 — Your First Lambda + API Gateway

Goal: a public HTTPS endpoint that runs a Python function. No EC2, no operating system, no scaling configuration.



The project structure

Just one Python file. No virtual env, no `requirements.txt` — this Lambda has no external dependencies.

```
hello-api/  
└─ lambda_function.py
```

Step 1.1 — Write the function locally

```
mkdir hello-api  
cd hello-api
```

File: `lambda_function.py`

```
import datetime  
  
# Simple HTML template rendered as the response.  
# Kept as a Python string so the project stays one file.  
PAGE = """<!doctype html>  
<html lang="en">  
<head>  
  <meta charset="utf-8">  
  <title>Hello from Lambda</title>  
  <style>  
    body {
```

```

    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
    margin: 0;
    background: linear-gradient(135deg, #232f3e 0%, #1a2536 100%);
    color: #f5f6f8;
    min-height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
}
.card {
    background: #ffffff;
    color: #1a1a1a;
    padding: 2.5rem 3rem;
    border-radius: 10px;
    box-shadow: 0 10px 40px rgba(0,0,0,0.3);
    max-width: 520px;
    width: 90%;
}
h1 {
    margin: 0 0 1rem 0;
    color: #232f3e;
    border-bottom: 3px solid #ff9900;
    padding-bottom: 0.6rem;
}
.row { margin: 0.6rem 0; }
.label { color: #5a6877; font-size: 0.9rem; }
.value { font-weight: 600; font-family: monospace; }
.badge {
    display: inline-block;
    background: #ff9900;
    color: #fff;
    padding: 2px 10px;
    border-radius: 12px;
    font-size: 0.85rem;
    font-weight: 700;
}
</style>
</head>
<body>
<div class="card">
  <h1>Hello from Lambda <span class="badge">serverless</span></h1>
  <p>__MESSAGE__</p>
  <div class="row">
    <span class="label">Served at (UTC):</span>
    <span class="value">__TIME__</span>
  </div>
  <div class="row">
    <span class="label">Tip:</span>
    try adding <code>?name=YourName</code> to the URL.
  </div>
</div>
</body>
</html>""

```

```

def lambda_handler(event, context):
    """The entrypoint Lambda calls for every request."""
    # Pull `name` from the query string, default to "World"
    qs = event.get("queryStringParameters") or {}
    name = qs.get("name", "World")

    message = f"Hello, {name}! This page is served by AWS Lambda."
    now = datetime.datetime.utcnow().isoformat() + "Z"

```

```
html = PAGE.replace("__MESSAGE__", message).replace("__TIME__", now)

return {
    "statusCode": 200,
    "headers": {"Content-Type": "text/html; charset=utf-8"},
    "body": html,
}
```

What is `event`? It is a dictionary containing everything about the request: HTTP method, path, query string, headers, body. The exact shape depends on what triggers the Lambda — for API Gateway HTTP APIs, it follows [this schema](#).

What is `context`? Information about the *Lambda runtime itself* — function name, remaining time, request ID. We rarely use it for simple functions.

Step 1.2 — Test the function locally

We can invoke the handler directly with a fake event and save the HTML output to a file we can open in the browser:

File: `test_local.py`

```
from lambda_function import lambda_handler

# Fake event matching API Gateway's HTTP API v2 shape
fake_event = {
    "requestContext": {"http": {"method": "GET"}},
    "queryStringParameters": {"name": "Ghana"},
}

response = lambda_handler(fake_event, None)

print("Status code:", response["statusCode"])
print("Content-Type:", response["headers"]["Content-Type"])

# Save the HTML so we can open it in the browser
with open("preview.html", "w") as f:
    f.write(response["body"])

print("Preview written to: preview.html")
```

Run it:

```
python test_local.py
```

You should see:

```
Status code: 200
Content-Type: text/html; charset=utf-8
Preview written to: preview.html
```

Open `preview.html` in your browser — you should see the styled card. Local test passed.

Step 1.3 — Package the function for upload

Lambda accepts a `.zip` of the function code. With no external dependencies, it is one file.

```
zip hello-api.zip lambda_function.py
ls -lh hello-api.zip
```

Step 1.4 — Create the Lambda function in the Console

In the AWS Console:

1. Make sure the Region in the top-right is **eu-west-1**
2. Search for **Lambda** -> open it
3. Click **Create function** (orange button, top right)
4. Choose **Author from scratch**
5. Function name: `hello-api`
6. Runtime: **Python 3.12**
7. Architecture: **x86_64**
8. Permissions: leave **Create a new role with basic Lambda permissions** (Lambda creates an IAM role for us that allows writing to CloudWatch Logs)
9. **Create function**

You land on the function page. Now upload our code:

1. Scroll to the **Code source** section
2. **Upload from** -> **.zip file** -> **Upload** -> select `hello-api.zip` -> **Save**

The file appears in the editor. Verify the **Handler** (top right of Code source panel) is `lambda_function.lambda_handler`. This tells Lambda: "in the file `lambda_function.py`, call the function `lambda_handler`".

Step 1.5 — Test the function from the Console

1. Click the **Test** tab (top of the Code source area)
2. Event name: `simple-event`
3. Event JSON: paste this:

```
{
  "requestContext": {"http": {"method": "GET"}},
  "queryStringParameters": {"name": "AmaliTech"}
}
```

4. **Save -> Test**

You should see green **"Execution result: succeeded"** and an HTML body in the response containing `Hello, AmaliTech!`. Lambda just ran our Python code in the cloud.

Step 1.6 — Add an API Gateway HTTP API in front

The Lambda works but is not reachable from the internet yet. We need an HTTPS endpoint.

In the AWS Console:

1. Search for **API Gateway** -> open it
2. **Create API** -> on the **HTTP API** tile click **Build**
3. **Integrations:** click **Add integration** -> **Lambda**
 - Lambda function: select **hello-api**
 - Version: **2.0**
4. API name: **hello-api**
5. **Next**
6. **Configure routes:**
 - Method: **ANY**
 - Resource path: **/hello**
 - Integration target: **hello-api**
7. **Next**
8. **Define stages:** leave the default **\$default** stage with auto-deploy on
9. **Next** -> **Create**

You will see the **Invoke URL** at the top, like **https://abc123xyz.execute-api.eu-west-1.amazonaws.com**. Copy it.

Step 1.7 — Test the live endpoint

Open the URL in your browser — the easy way:

Open **https://<your-invoke-url>/hello** in your browser. Also try **https://<your-invoke-url>/hello?name=Ghana**.

You should see the styled card.

From the terminal you can also confirm the response:

```
API_URL=https://<your-invoke-url>

# Default name
curl "$API_URL/hello"

# Pass a name
curl "$API_URL/hello?name=Ghana"
```

You will see raw HTML on the terminal — paste a few bytes of it into a browser if you want to confirm, but the easiest test is just opening the URL.

Your Python function is live on the public internet, with HTTPS, served from a real domain — no servers anywhere.

Step 1.8 — Inspect the logs

Every **print()** and every uncaught exception goes to CloudWatch Logs automatically.

1. In the Lambda Console, open `hello-api`
2. **Monitor** tab -> **View CloudWatch logs**
3. Click the most recent log stream
4. You see your function's stdout, plus AWS-added lines for start/end of each invocation

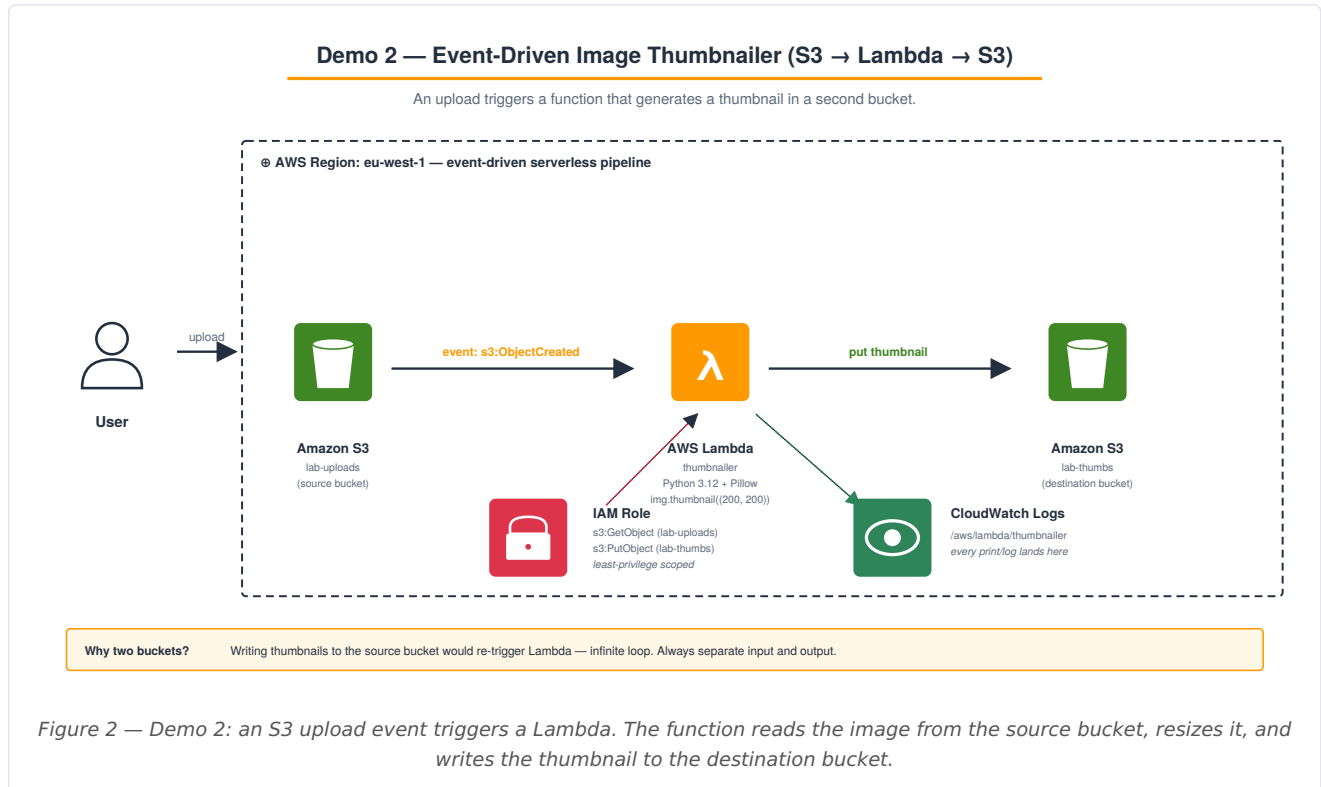
What we built in Demo 1

A public Python API. Total cost so far: \$0.00. Lambda's free tier covers 1 million invocations per month.

Keep this Lambda — we will not need it for Demo 2 or 3, but you can delete it during Cleanup.

Demo 2 — Event-Driven Image Thumbnailer

Goal: when someone uploads an image to an S3 bucket, a Lambda automatically generates a 200x200 thumbnail in a second bucket. No HTTP, no API — pure event-driven serverless.



The project structure

This Lambda has a real Python dependency (**Pillow**, the image library), so the zip is more than one file.

```
thumbnailer/
├─ lambda_function.py    # The thumbnail code
└─ build/                # We pip-install Pillow here, then zip it
```

Step 2.1 — Write the function

```
cd ~    # or wherever you keep projects
mkdir thumbnailer
cd thumbnailer
```

File: `lambda_function.py`

```

import os
import boto3
from PIL import Image
from io import BytesIO

s3 = boto3.client("s3")
DEST_BUCKET = os.environ["DEST_BUCKET"]
THUMB_SIZE = (200, 200)

def lambda_handler(event, context):
    # S3 events can contain multiple records; iterate over each
    for record in event["Records"]:
        src_bucket = record["s3"]["bucket"]["name"]
        src_key = record["s3"]["object"]["key"]

        print(f"Processing s3://{src_bucket}/{src_key}")

        # 1. Download the image bytes from the source bucket
        obj = s3.get_object(Bucket=src_bucket, Key=src_key)
        image_bytes = obj["Body"].read()

        # 2. Resize using Pillow
        with Image.open(BytesIO(image_bytes)) as img:
            img.thumbnail(THUMB_SIZE)
            buffer = BytesIO()
            # Preserve format (JPEG, PNG, etc.)
            fmt = img.format or "JPEG"
            img.save(buffer, format=fmt)
            buffer.seek(0)

        # 3. Upload the thumbnail to the destination bucket
        dst_key = f"thumb-{src_key}"
        s3.put_object(
            Bucket=DEST_BUCKET,
            Key=dst_key,
            Body=buffer,
            ContentType=obj["ContentType"]
        )
        print(f"Wrote thumbnail to s3://{DEST_BUCKET}/{dst_key}")

    return {"status": "ok"}

```

Why `os.environ["DEST_BUCKET"]` ? We pass the destination bucket name as a **Lambda environment variable** — that way the code stays the same even if the bucket name changes.

Why iterate over `event["Records"]` ? S3 can batch multiple object events into one invocation. Always handle the list, not just `[0]`.

Step 2.2 — Test locally with Pillow installed

```

# Create a venv and install Pillow + boto3
python3 -m venv venv
source venv/bin/activate
pip install Pillow boto3

```

We cannot fully test without an S3 bucket, but we can verify the imports and Pillow:

File: `test_local.py`

```

from PIL import Image
print("Pillow version:", Image.__version__)
# Smoke test: open and resize a tiny image
img = Image.new("RGB", (1000, 1000), color="orange")
img.thumbnail((200, 200))
print("Resized to:", img.size)

```

Run:

```

python test_local.py
# Expect: Pillow version: 10.x.x and Resized to: (200, 200)

```

Step 2.3 — Package the function with Pillow inside

Lambda needs Pillow bundled into the zip. Because your Ubuntu laptop is `x86_64` (same architecture as Lambda's default runtime), we can install Pillow directly with `pip` and bundle it.

```

# Create a build folder and install Pillow into it
mkdir -p build
pip install Pillow -t build/

# Copy the function code into the build folder
cp lambda_function.py build/

# Zip everything from the build folder
cd build
zip -r ../thumbnailer.zip .
cd ..

ls -lh thumbnailer.zip

```

Note: if your laptop is ARM (Apple Silicon, Raspberry Pi, etc.) and the Lambda is x86, this trick fails — Pillow contains compiled C code. In that case, build inside a Linux x86 container: `docker run --rm --platform linux/amd64 -v "$PWD/build":/var/task public.ecr.aws/sam/build-python3.12 pip install Pillow -t /var/task`

Step 2.4 — Create the two S3 buckets

In the AWS Console:

1. **S3 -> Create bucket**
2. Bucket name: `lab-uploads-<your-initials>-<random-number>` (must be globally unique). The author of this guide uses `lab-uploads-cnm-9472` — pick something similar with **your** initials and a random number.
3. AWS Region: **eu-west-1**
4. Leave defaults (Block public access stays on)
5. Tags: **Project = python-chapter-demo**
6. **Create bucket**

Then **repeat** with a second bucket: `lab-thumbs-<your-initials>-<random-number>` — using the **same suffix** as the first (e.g. `lab-thumbs-cnm-9472`). Keeping the suffix consistent makes the IAM policy and environment variable easier to manage.

Why two buckets? If thumbnails went back into the source bucket, each `put_object` would trigger Lambda again — **infinite loop**. Source and destination must be different.

Step 2.5 — Create an IAM role for the Lambda

The Lambda needs permission to read from source bucket and write to destination bucket.

In the AWS Console:

1. **IAM** -> **Roles** -> **Create role**
2. Trusted entity: **AWS service**
3. Use case: **Lambda** -> **Next**
4. Permissions: attach `AWSLambdaBasicExecutionRole` (only — we add the S3 permissions inline next)
5. Role name: `ThumbnailerLambdaRole`
6. **Create role**

Now add an inline policy for the buckets:

1. Open `ThumbnailerLambdaRole` -> **Add permissions** -> **Create inline policy**
2. Click the **JSON** tab and paste:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::lab-uploads-cnm-9472/*"
    },
    {
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::lab-thumbs-cnm-9472/*"
    }
  ]
}
```

Replace `cnm-9472` with the suffix you chose in Step 2.4.

3. **Next** -> Policy name: `ThumbnailerS3Access` -> **Create policy**

Why "least-privilege"? This role can read from exactly one bucket and write to exactly one bucket. If our function code were ever exploited, the attacker could not delete other buckets, list other resources, or do anything else.

Step 2.6 — Create the Lambda function

In the AWS Console:

1. Lambda -> Create function

2. Function name: `thumbnailer`

3. Runtime: **Python 3.12**

4. **Change default execution role -> Use an existing role -> select** `ThumbnailerLambdaRole`

5. Create function

Upload our zip:

6. **Code source -> Upload from -> .zip file -> select** `thumbnailer.zip` -> **Save**

Set the environment variable:

7. **Configuration** tab -> **Environment variables** -> **Edit** -> **Add environment variable:**

- Key: `DEST_BUCKET`
- Value: `lab-thumbs-cnm-9472` (use your own suffix)

8. Save

Increase memory and timeout (image processing needs more than the defaults):

9. **Configuration** tab -> **General configuration** -> **Edit**

10. Memory: `512` MB

11. Timeout: `30` seconds

12. Save

Step 2.7 — Wire the S3 bucket to the Lambda

The Lambda needs to be **triggered** by uploads to the source bucket.

In the AWS Console:

1. Still on the `thumbnailer` function page -> **Add trigger** (top of the function diagram)

2. Source: **S3**

3. Bucket: select `lab-uploads-cnm-9472` (your source bucket from Step 2.4)

4. Event types: **All object create events**

5. Suffix (optional): `.jpg` (so we only trigger for JPEGs — try also `.png` for variety)

6. Tick **I acknowledge that using the same S3 bucket for both input and output is not recommended** (we are NOT using the same bucket, so this is fine — the warning is generic)

7. Add

Step 2.8 — Test by uploading an image

Find any JPEG on your laptop (or download a sample). Then:

In the AWS Console:

1. **S3** -> open `lab-uploads-cnm-9472` -> **Upload** -> **Add files** -> select a `.jpg`

2. Upload

Within 2-3 seconds, the Lambda triggers automatically. Check the result:

1. **S3** -> open `lab-thumbs-cnm-9472`
2. You should see a file named `thumb-<original-filename>.jpg`
3. Click it -> **Open** to view the 200x200 thumbnail

Inspect the logs to see what happened:

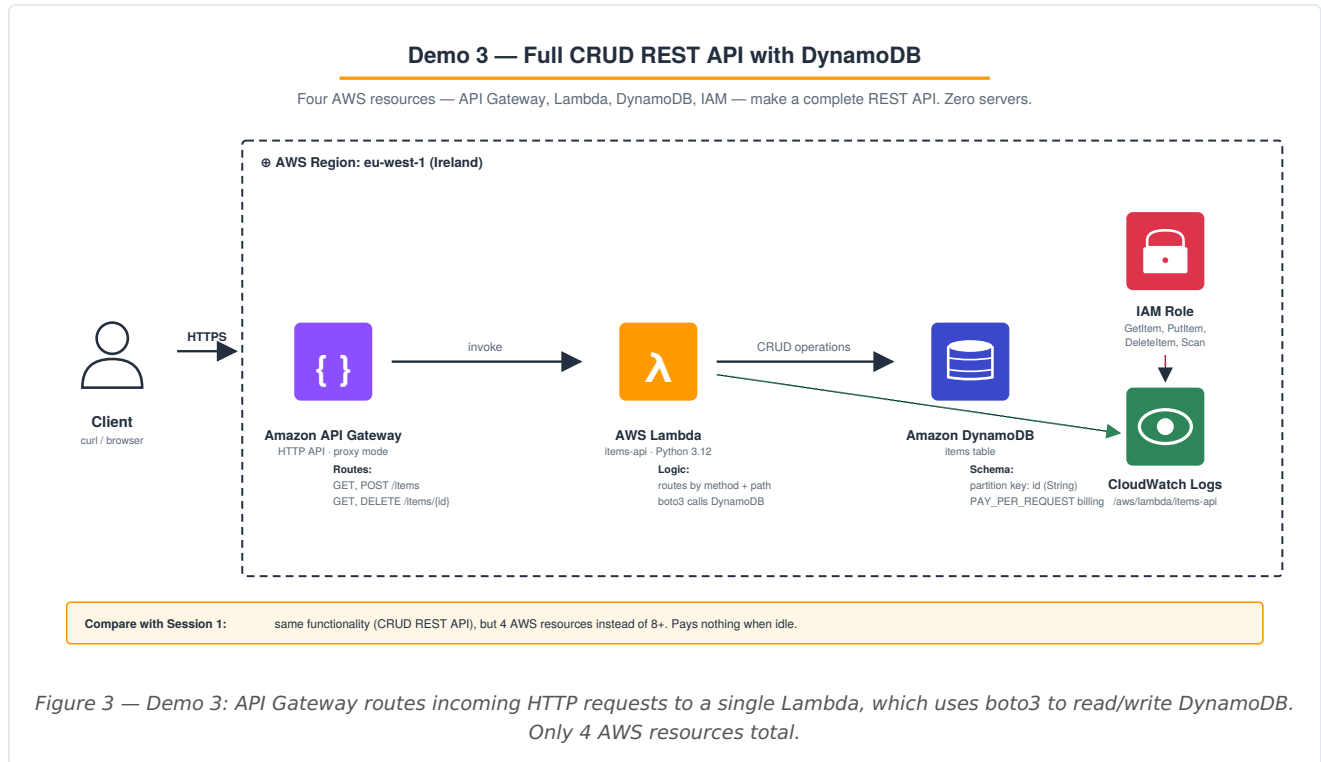
1. **Lambda** -> `thumbnailer` -> **Monitor** -> **View CloudWatch logs**
2. Latest log stream -> you see `Processing s3://lab-uploads...` and `Wrote thumbnail to s3://lab-thumbs...`

What we built in Demo 2

A self-triggering image-processing pipeline. To use it: drop a file in S3. That is the whole interface. Scale is automatic — drop 1000 images and 1000 Lambdas run in parallel.

Demo 3 — Full CRUD REST API with DynamoDB

Goal: a complete REST API with Create, Read, and Delete operations, backed by a NoSQL database. Compare with Session 1's Demo 2 — same functionality, far fewer resources, pays nothing when idle.



The project structure

```
items-api/
└─ lambda_function.py
```

No external dependencies — **boto3** is pre-installed in the Lambda runtime.

Step 3.1 — Write the function

```
mkdir items-api
cd items-api
```

File: **lambda_function.py**

```
import json
import os
import uuid
import boto3

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table(os.environ["TABLE_NAME"])

# ----- Index page (HTML) -----
```



```

INDEX_HTML = """<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Items API</title>
  <style>
    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
      margin: 0;
      background: linear-gradient(135deg, #232f3e 0%, #1a2536 100%);
      color: #f5f6f8;
      min-height: 100vh;
      display: flex;
      align-items: center;
      justify-content: center;
    }
    .card {
      background: #ffffff;
      color: #1a1a1a;
      padding: 2.5rem 3rem;
      border-radius: 10px;
      box-shadow: 0 10px 40px rgba(0,0,0,0.3);
      max-width: 600px;
      width: 90%;
    }
    h1 {
      margin: 0 0 1rem 0;
      color: #232f3e;
      border-bottom: 3px solid #ff9900;
      padding-bottom: 0.6rem;
    }
    table { width: 100%; border-collapse: collapse; margin-top: 1rem; }
    th, td { padding: 8px 10px; text-align: left; border-bottom: 1px solid #eee; }
    th { background: #fafbfd; color: #5a6877; font-weight: 600; }
    code { background: #fff4dc; color: #8a4500; padding: 1px 6px; border-radius: 3px; }
  </style>
</head>
<body>
  <div class="card">
    <h1>Items API</h1>
    <p>A serverless CRUD API backed by DynamoDB. Available endpoints:</p>
    <table>
      <tr><th>Method</th><th>Path</th><th>Description</th></tr>
      <tr><td><code>GET</code></td><td><code>/items</code></td><td>List all items</td></tr>
      <tr><td><code>POST</code></td><td><code>/items</code></td><td>Create an item</td></tr>
      <tr><td><code>GET</code></td><td><code>/items/{id}</code></td><td>Fetch one item</td></tr>
      <tr><td><code>DELETE</code></td><td><code>/items/{id}</code></td><td>Delete one item</td></tr>
    </table>
  </div>
</body>
</html>"""

```

----- Helpers -----

```

def respond_json(status_code, body):
    """JSON response for API calls."""
    return {
        "statusCode": status_code,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps(body),
    }

```

```

def respond_html(status_code, html):
    """HTML response for the index page."""
    return {
        "statusCode": status_code,
        "headers": {"Content-Type": "text/html; charset=utf-8"},
        "body": html,
    }

# ----- Route handlers -----

def list_items():
    """GET /items - return all items."""
    result = table.scan()
    return respond_json(200, result.get("Items", []))

def create_item(body):
    """POST /items - create a new item and return it."""
    if not body or "name" not in body:
        return respond_json(400, {"error": "Field 'name' is required"})
    item = {"id": str(uuid.uuid4()), "name": body["name"]}
    table.put_item(Item=item)
    return respond_json(201, item)

def get_item(item_id):
    """GET /items/{id} - return one item or 404."""
    result = table.get_item(Key={"id": item_id})
    item = result.get("Item")
    if not item:
        return respond_json(404, {"error": "Not found"})
    return respond_json(200, item)

def delete_item(item_id):
    """DELETE /items/{id} - delete one item."""
    table.delete_item(Key={"id": item_id})
    return respond_json(204, {})

# ----- The entry point -----

def lambda_handler(event, context):
    """Route the request to the right handler based on method + path."""
    method = event["requestContext"]["http"]["method"]
    path = event["requestContext"]["http"]["path"]

    # Parse the JSON body, if any
    body = {}
    if event.get("body"):
        try:
            body = json.loads(event["body"])
        except json.JSONDecodeError:
            return respond_json(400, {"error": "Invalid JSON"})

    # Path parameter (the {id} part)
    item_id = (event.get("pathParameters") or {}).get("id")

    # Routes
    if method == "GET" and path == "/":
        return respond_html(200, INDEX_HTML)
    if method == "GET" and path == "/items":

```

```

        return list_items()
    if method == "POST" and path == "/items":
        return create_item(body)
    if method == "GET" and item_id:
        return get_item(item_id)
    if method == "DELETE" and item_id:
        return delete_item(item_id)

    return respond_json(404, {"error": "Route not found"})

```

Why one Lambda for all routes? It is simpler for small APIs. For larger APIs you would split each route into its own function. Both patterns are valid.

What is `uuid.uuid4()`? It generates a random unique ID like `2f1b3...c8a`. DynamoDB requires a primary key — we use the UUID as the `id` attribute.

Step 3.2 — Package the function

No dependencies, so it is a one-file zip:

```
zip items-api.zip lambda_function.py
```

Step 3.3 — Create the DynamoDB table

In the AWS Console:

1. Search for **DynamoDB** -> open it
2. **Tables** -> **Create table**
3. Table name: `items`
4. Partition key: `id`, type **String**
5. **Table settings:** select **Customize settings**
6. **Capacity mode:** **On-demand** (pay per request, no upfront capacity)
7. Leave everything else at default
8. Tags: `Project = python-chapter-demo`
9. **Create table**

What is a partition key? DynamoDB's primary key. Every item needs a unique value for `id`. To fetch an item you need to know its `id` — DynamoDB looks items up by partition key in constant time.

Why on-demand? No capacity to plan. You pay \$0 when no one uses the API and a tiny amount per request when traffic comes. Perfect for unpredictable workloads.

Step 3.4 — Create the IAM role

In the AWS Console:

1. **IAM -> Roles -> Create role**
2. Trusted entity: **AWS service**, Use case: **Lambda -> Next**
3. Attach **AWSLambdaBasicExecutionRole**
4. Role name: **ItemsApiLambdaRole**
5. **Create role**

Add a DynamoDB inline policy:

6. Open **ItemsApiLambdaRole** -> **Add permissions -> Create inline policy -> JSON:**

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "dynamodb:GetItem",
      "dynamodb:PutItem",
      "dynamodb>DeleteItem",
      "dynamodb:Scan"
    ],
    "Resource": "arn:aws:dynamodb:eu-west-1:*:table/items"
  }]
}
```

7. **Next** -> name: **ItemsApiDynamoDBAccess** -> **Create policy**

Step 3.5 — Create the Lambda function

In the AWS Console:

1. **Lambda -> Create function**
2. Function name: **items-api**
3. Runtime: **Python 3.12**
4. **Change default execution role -> Use an existing role -> select** **ItemsApiLambdaRole**
5. **Create function**

Upload code:

6. **Code source -> Upload from -> .zip file ->** **items-api.zip** -> **Save**

Set environment variable:

7. **Configuration -> Environment variables -> Edit -> add:**
 - Key: **TABLE_NAME**
 - Value: **items**
8. **Save**

Important — write down your function name (`items-api`). You will use it in two places later if you do Section A:

1. **Local `.env`** as the value of `FUNCTION_NAME`.
2. **GitHub Actions Secrets** as the value of the `FUNCTION_NAME` secret (so the pipeline knows which Lambda to update).

Both must match the actual Lambda function name **exactly** — a typo breaks the deployment. The same applies to your AWS Account ID and Region: any value the pipeline needs to talk to AWS goes into both `.env` and GitHub Secrets.

Step 3.6 — Create the API Gateway

In the AWS Console:

1. **API Gateway -> Create API -> HTTP API -> Build**
2. **Add integration -> Lambda -> select `items-api` (version 2.0)**
3. API name: `items-api`
4. **Next**
5. **Configure routes:** we need three routes
 - Route 1: method `GET`, path `/`, integration `items-api`
 - Click **Add route:**
 - Route 2: method `ANY`, path `/items`, integration `items-api`
 - Click **Add route:**
 - Route 3: method `ANY`, path `/items/{id}`, integration `items-api`
6. **Next -> default stage -> Next -> Create**

Copy the **Invoke URL** (e.g. `https://xyz.execute-api.eu-west-1.amazonaws.com`).

Step 3.7 — Test the API

Open the base URL in your **browser** first:

Open `https://<your-invoke-url>/` — you should see the styled "Items API" index page listing the endpoints.

Then exercise the API from your terminal:

```

API=https://<your-invoke-url>

# CREATE – POST /items
curl -X POST "$API/items" \
  -H "Content-Type: application/json" \
  -d '{"name": "First item"}'
# Response: {"id": "...", "name": "First item"}

# Save the id for later
ITEM_ID=<copy-the-id-from-the-response-above>

# Create a second item
curl -X POST "$API/items" \
  -H "Content-Type: application/json" \
  -d '{"name": "Second item"}'

# READ – GET /items
curl "$API/items"
# Response: [{...}, {...}]

# READ – GET /items/{id}
curl "$API/items/$ITEM_ID"

# DELETE – DELETE /items/{id}
curl -X DELETE "$API/items/$ITEM_ID"

# Verify it is gone
curl "$API/items/$ITEM_ID"
# Response: {"error": "Not found"}

```

A complete REST API in 4 AWS resources (Lambda + API Gateway + DynamoDB + IAM Role). No EC2, no load balancer, no Auto Scaling Group, no Security Groups, no VPC.

Step 3.8 — Inspect the database

In the AWS Console:

1. **DynamoDB** -> **Tables** -> **items** -> **Explore table items**
2. You see the items we created, with their IDs and names

What we built in Demo 3

A complete production-ready REST API. Total monthly cost when idle: \$0. Cost per request: a fraction of a cent.

The comparison with Session 1 is striking:

	Session 1 (server-based)	Session 2 (serverless)
Resources to create	8+ (VPC, SGs, ALB, TG, LT, ASG, EC2 ×2, ...)	4 (Lambda, API GW, DynamoDB, IAM)
Time to deploy	25–30 minutes	5–10 minutes
Cost when idle	\$30+/month (EC2 instances + ALB)	\$0
Cost per request	Already paid	\$0.0000002
What you manage	OS, runtime, scaling	Code, permissions

Both patterns are valid. Choose based on traffic shape and how much control you need.

Cleanup — Delete everything we created

In the AWS Console:

1. **API Gateway** -> delete `hello-api` , `items-api`
2. **Lambda** -> delete `hello-api` , `thumbnailer` , `items-api`
3. **DynamoDB** -> **Tables** -> select `items` -> **Delete table**
4. **S3** -> open `lab-uploads-cnm-9472` -> **Empty** -> **Delete bucket**; same for `lab-thumbs-cnm-9472` (use the suffixes you chose)
5. **IAM** -> **Roles** -> delete `ThumbnailerLambdaRole` , `ItemsApiLambdaRole` , and any Lambda-created roles (named `hello-api-role-...`)
6. **CloudWatch** -> **Log groups** -> delete `/aws/lambda/hello-api` , `/aws/lambda/thumbnailer` , `/aws/lambda/items-api` (optional — they cost almost nothing)

Section A — CI/CD with GitHub Actions (optional, ~15 minutes)

Goal: every push to `main` automatically packages the Lambda and updates the function in AWS.

What we add

```
items-api/
├─ lambda_function.py
├─ .env.example      # NEW: template showing required variables
├─ .gitignore        # NEW: ignore .env so secrets never leak
├─ .github/
│   └─ workflows/
│       └─ deploy.yml # NEW: the CI/CD workflow
```

Important — secrets management rule: account IDs, role ARNs, and similar values must **never** be hardcoded in tracked files. They go in two places:

- **Local development:** a `.env` file on your laptop. This file is `.gitignored` and never reaches GitHub.
- **CI/CD pipeline: GitHub Actions Secrets.** The workflow reads them at runtime with `${{ secrets.NAME }}`.

Both places get filled from the same `.env.example` template, which IS committed — but with placeholder values, not real ones.

Step A.1 — Set up the GitHub Actions IAM role (one-time)

If you did Session 1 Section A, you already have an OIDC provider — reuse it. Otherwise:

In the AWS Console:

1. **IAM -> Identity providers -> Add provider**
2. Provider type: **OpenID Connect**
3. Provider URL: `https://token.actions.githubusercontent.com`
4. Audience: `sts.amazonaws.com`
5. **Add provider**

Create a role for the Lambda deployment:

1. **IAM -> Roles -> Create role**
2. Trusted entity: **Web identity** -> provider: `token.actions.githubusercontent.com`, audience: `sts.amazonaws.com`
3. GitHub organization: `<your-username>`, repository: `items-api`, branch: `main`
4. **Next**
5. Attach `AWSLambda_FullAccess` (scope down in production)
6. Role name: `GitHubActionsLambdaRole`
7. **Create role**

Copy the role ARN.

Step A.2 — Push the project to GitHub

```
cd items-api
git init
git add .
git commit -m "Initial commit"

# Create a public repo on github.com/new called "items-api"
git branch -M main
git remote add origin https://github.com/<your-username>/items-api.git
git push -u origin main
```

Step A.3 — Create `.env.example` and your local `.env`

The `.env.example` file documents which configuration values the project needs. It is committed; the real `.env` is not.

File: `.env.example`

```
# AWS region where the Lambda lives
AWS_REGION=eu-west-1

# Your 12-digit AWS Account ID (top-right of the AWS Console)
AWS_ACCOUNT_ID=123456789012

# Name of the IAM role GitHub Actions assumes via OIDC
AWS_ROLE_NAME=GitHubActionsLambdaRole

# Lambda function name
FUNCTION_NAME=items-api
```

Copy it to a real `.env` with **your actual values**:

```
cp .env.example .env

# Edit .env with your real values
nano .env
# Replace AWS_ACCOUNT_ID with your 12-digit account ID
```

Make sure `.env` is gitignored so it never reaches GitHub:

File: `.gitignore`

```
*.zip
.env
__pycache__/
```

You can use the file locally for manual deployments:

```
# Load .env into the current shell
set -a; source .env; set +a

# Example: deploy manually with the same values the pipeline will use
zip function.zip lambda_function.py
aws lambda update-function-code \
  --function-name "$FUNCTION_NAME" \
  --zip-file fileb://function.zip \
  --region "$AWS_REGION"
```

Step A.4 — Add the same values to GitHub Secrets

The CI/CD workflow runs on GitHub's servers and cannot read your local `.env`. We mirror the same values into GitHub Actions Secrets.

On GitHub (in your browser):

1. Open your repository: `https://github.com/<your-username>/items-api`
2. Click **Settings** (top tab, far right)
3. In the left sidebar, expand **Secrets and variables** -> click **Actions**
4. Click **New repository secret** (green button, top right) and add each entry from your `.env` file:

Secret name	Value
<code>AWS_REGION</code>	<code>eu-west-1</code>
<code>AWS_ACCOUNT_ID</code>	your 12-digit AWS account ID
<code>AWS_ROLE_NAME</code>	<code>GitHubActionsLambdaRole</code>
<code>FUNCTION_NAME</code>	<code>items-api</code>

Click **Add secret** after each. You should end up with **four** secrets.

Tip: secret names should match the `.env` keys exactly — that makes the workflow file easy to read and the values easy to mirror.

Step A.5 — Add the workflow file

```
mkdir -p .github/workflows
```

File: `.github/workflows/deploy.yml`

```

name: Deploy Lambda

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Run import-check (smoke test)
        run: python -c "import lambda_function; print('OK!)"

      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::${{ secrets.AWS_ACCOUNT_ID }}:role/${{ secrets.AWS_ROLE_NAME }}
          aws-region: ${{ secrets.AWS_REGION }}

      - name: Package the function
        run: zip function.zip lambda_function.py

      - name: Update Lambda code
        run: |
          aws lambda update-function-code \
            --function-name ${{ secrets.FUNCTION_NAME }} \
            --zip-file fileb://function.zip

```

Notice: there are **no hardcoded values** in this file — every `${{ secrets.XXX }}` reference pulls from GitHub Secrets. The same workflow file would work for any team member who forks the repo and sets their own secrets.

Step A.6 — Push and watch it deploy

```

git add .env.example .gitignore .github/
git commit -m "Add deploy workflow with secret-driven config"
git push origin main

```

Verify before pushing: run `git status` and confirm `.env` does **not** appear in the list of tracked files. If it does, your `.gitignore` is wrong and your account ID is about to be public. Stop and fix it before pushing.

Open the **Actions** tab on GitHub. After ~30 seconds you should see the workflow finish green, and the Lambda's code is now whatever was on `main`.

From now on, `git push` is your deploy command. For serverless, that is the whole CI/CD pipeline. To rotate any

value (move regions, rename the function, switch role), update the corresponding GitHub Secret — no code change required.